Set-up environment

This example is takem from https://github.com/NielsRogge/Transformers-Tutorials/blob/master/LayoutLMv3/Fine_tune_LayoutLMv3_on_FUNSD_(HuggingFace_Trainer).ipynb

I had to modify few lines, but the essential theory is same. So you can read above for the theory. First, we install 🤗 Transformers, as well as 🤗 Datasets and Seqeval (the latter is useful for evaluation metrics such as F1 on sequence labeling tasks).

```
!pip install -q git+https://github.com/huggingface/transformers.git
```

```
  Installing build dependencies ... done
  Getting requirements to build wheel ... done
  Preparing metadata (pyproject.toml) ... done
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 2.9/2.9 MB 48.6 MB/s eta 0:00:00
  Building wheel for transformers (pyproject.toml) ... done
```

```
!pip install evaluate # Install the evaluate package
!pip install -q datasets seqeval
!pip install --upgrade datasets seqeval # make sure the latest version of datasets is installed
```

```
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.10/dist-packages (from pandas->evaluate) (2024.2
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.10/dist-packages (from pandas->evaluate) (2024
Requirement already satisfied: aiohappyeyeballs>=2.3.0 in /usr/local/lib/python3.10/dist-packages (from aiohttp->datase
Requirement already satisfied: aiosignal>=1.1.2 in /usr/local/lib/python3.10/dist-packages (from aiohttp->datasets>=2.6
Requirement already satisfied: attrs>=17.3.0 in /usr/local/lib/python3.10/dist-packages (from aiohttp->datasets>=2.0.0
Requirement already satisfied: frozenlist>=1.1.1 in /usr/local/lib/python3.10/dist-packages (from aiohttp->datasets>=2
Requirement already satisfied: multidict<7.0,>=4.5 in /usr/local/lib/python3.10/dist-packages (from aiohttp->datasets>=
Requirement already satisfied: yarl<2.0,>=1.12.0 in /usr/local/lib/python3.10/dist-packages (from aiohttp->datasets>=2
Requirement already satisfied: async-timeout<5.0,>=4.0 in /usr/local/lib/python3.10/dist-packages (from aiohttp->datase
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.10/dist-packages (from python-dateutil>=2.8.2->pandas
Downloading evaluate-0.4.3-py3-none-any.whl (84 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 84.0/84.0 kB 9.4 MB/s eta 0:00:00
Downloading datasets-3.0.1-py3-none-any.whl (471 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 471.6/471.6 kB 32.3 MB/s eta 0:00:00
Downloading dill-0.3.8-py3-none-any.whl (116 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 116.3/116.3 kB 12.7 MB/s eta 0:00:00
Downloading multiprocess-0.70.16-py310-none-any.whl (134 kB)
```

```
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.10/dist-packages (from requests>=2.32
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.10/dist-packages (from requests>=2.32.2->datasets
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.10/dist-packages (from requests>=2.32.2->da
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.10/dist-packages (from requests>=2.32.2->da
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.10/dist-packages (from scikit-learn>=0.21.3->seqe
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.10/dist-packages (from scikit-learn>=0.21.3->sec
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.10/dist-packages (from scikit-learn>=0.21
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.10/dist-packages (from pandas->dataset
```

Load dataset

Next, we load a dataset from the 🤗 hub. This one is the FUNSD dataset, a collection of annotated forms.

```
from datasets import load_dataset

# this dataset uses the new Image feature :)
dataset = load_dataset("nielsr/funsd-layoutlmv3")
```

```
/usr/local/lib/python3.10/dist-packages/huggingface_hub/utils/_token.py:89: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens),
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
0000.parquet: 100%  26.3M/26.3M [00:00<00:00, 80.2MB/s]
0000.parquet: 100%  9.54M/9.54M [00:00<00:00, 166MB/s]
Generating train split: 100%  149/149 [00:00<00:00, 938.75 examples/s]
Generating test split: 100%  50/50 [00:00<00:00, 854.16 examples/s]
```

As we can see, the dataset consists of 2 splits ("train" and "test"), and each example contains a list of words ("tokens") with corresponding boxes ("bboxes"), and the words are tagged ("ner_tags"). Each example also include the original image ("image").

```
dataset
```

```
DatasetDict({
    train: Dataset({
        features: ['id', 'tokens', 'bboxes', 'ner_tags', 'image'],
        num_rows: 149
    })
    test: Dataset({
        features: ['id', 'tokens', 'bboxes', 'ner_tags', 'image'],
        num_rows: 50
    })
})
```

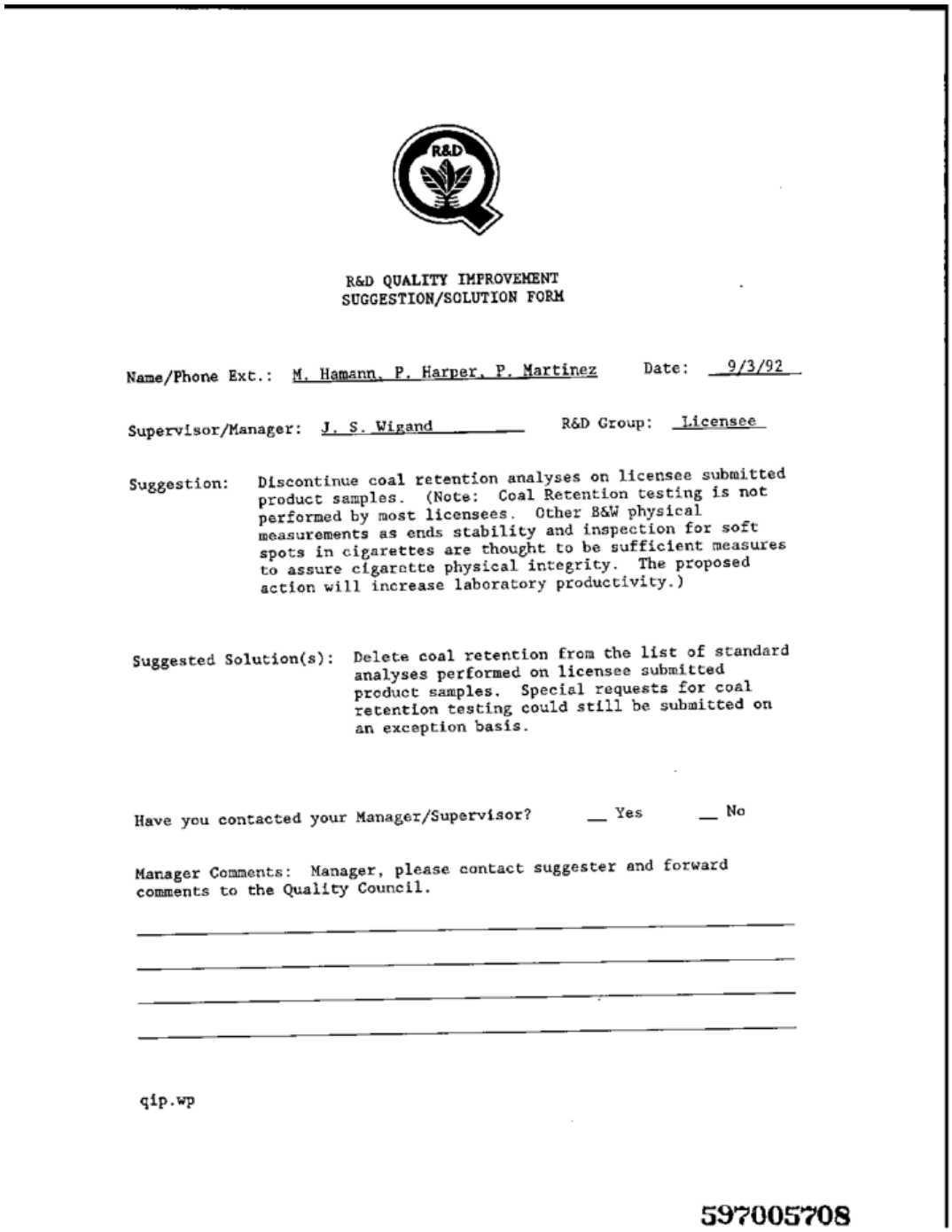
Let's check the features:

```
dataset["train"].features
```

```
{'bboxes': Sequence(feature=Sequence(feature=Value(dtype='int64', id=None), length=-1, id=None), length=-1, id=None),
 'id': Value(dtype='string', id=None),
 'image': Image(decode=True, id=None),
 'ner_tags': Sequence(feature=ClassLabel(num_classes=7, names=['O', 'B-HEADER', 'I-HEADER', 'B-QUESTION', 'I-QUESTION',
'B-ANSWER', 'I-ANSWER'], id=None), length=-1, id=None),
 'tokens': Sequence(feature=Value(dtype='string', id=None), length=-1, id=None)}
```

Note that you can directly see the example in a notebook (as the "image" column is of type Image).

```
example = dataset["train"][0]
example["image"]
```



R&D QUALITY IMPROVEMENT
SUGGESTION/SOLUTION FORM

Name/Phone Ext.: M. Hamann, P. Harper, P. Martinez Date: 9/3/92

Supervisor/Manager: J. S. Wigand R&D Group: Licensee

Suggestion: Discontinue coal retention analyses on licensee submitted product samples. (Note: Coal Retention testing is not performed by most licensees. Other B&W physical measurements as ends stability and inspection for soft spots in cigarettes are thought to be sufficient measures to assure cigarette physical integrity. The proposed action will increase laboratory productivity.)

Suggested Solution(s): Delete coal retention from the list of standard analyses performed on licensee submitted product samples. Special requests for coal retention testing could still be submitted on an exception basis.

Have you contacted your Manager/Supervisor? __ Yes __ No

Manager Comments: Manager, please contact suggester and forward comments to the Quality Council.

qip.wp

597005708

```
words, boxes, ner_tags = example["tokens"], example["bboxes"], example["ner_tags"]
print("words=", words)
print("len(words)=",len(words))
print("boxes=", boxes)
print("len(boxes)=",len(boxes))
print("ner_tags=", ner_tags)
print("len(ner_tags)=",len(ner_tags))
```

```
words= ['R&D', ':', 'Suggestion:', 'Date:', 'Licensee', 'Yes', 'No', '597005708', 'R&D', 'QUALITY', 'IMPROVEMENT', 'SUGGE
len(words)= 145
boxes= [[383, 91, 493, 175], [287, 316, 295, 327], [124, 355, 221, 370], [632, 268, 679, 282], [670, 309, 748, 323], [604
len(boxes)= 145
```

```
ner_tags= [0, 3, 3, 3, 5, 3, 3, 0, 1, 2, 2, 2, 2, 2, 3, 4, 4, 4, 4, 5, 6, 6, 6, 6, 6, 5, 6, 6, 3, 4, 5, 6, 6, 3, 4, 4, 5,
len(ner_tags)= 145
```

Prepare dataset

Next, we prepare the dataset for the model. This can be done very easily using `LayoutLMv3Processor`, which internally wraps a `LayoutLMv3FeatureExtractor` (for the image modality) and a `LayoutLMv3Tokenizer` (for the text modality) into one.

Basically, the processor does the following internally:

- the feature extractor is used to resize + normalize each document image into `pixel_values`
- the tokenizer is used to turn the words, boxes and NER tags into token-level `input_ids`, `attention_mask` and `labels`.

The processor simply returns a dictionary that contains all these keys.

```
from transformers import AutoProcessor

# we'll use the Auto API here - it will load LayoutLMv3Processor behind the scenes,
# based on the checkpoint we provide from the hub
processor = AutoProcessor.from_pretrained("microsoft/layoutlmv3-base", apply_ocr=False)
```

```
The cache for model files in Transformers v4.22.0 has been updated. Migrating your old cache. This is a one-time only ope
0/0 [00:00<?, ?it/s]

preprocessor_config.json: 100%  275/275 [00:00<00:00, 10.4kB/s]

tokenizer_config.json: 100%  1.14k/1.14k [00:00<00:00, 77.0kB/s]

config.json: 100%  856/856 [00:00<00:00, 56.6kB/s]

vocab.json: 100%  899k/899k [00:01<00:00, 863kB/s]

merges.txt: 100%  456k/456k [00:00<00:00, 2.08MB/s]
```

We'll first create `id2label` and label2id mappings, useful for inference. Note that `LayoutLMv3ForTokenClassification` (the model we'll use later on) will simply output an integer index for a particular class (for each token), so we still need to map it to an actual class name.

```
from datasets.features import ClassLabel

features = dataset["train"].features
column_names = dataset["train"].column_names
image_column_name = "image"
text_column_name = "tokens"
boxes_column_name = "bboxes"
label_column_name = "ner_tags"

# In the event the labels are not a `Sequence[ClassLabel]`, we will need to go through the dataset to get the
# unique labels.
def get_label_list(labels):
    unique_labels = set()
    for label in labels:
        unique_labels = unique_labels | set(label)
    label_list = list(unique_labels)
    label_list.sort()
    return label_list

if isinstance(features[label_column_name].feature, ClassLabel):
    label_list = features[label_column_name].feature.names
    # No need to convert the labels since they are already ints.
    id2label = {k: v for k,v in enumerate(label_list)}
    label2id = {v: k for k,v in enumerate(label_list)}
else:
```

```
    label_list = get_label_list(dataset["train"][label_column_name])
    id2label = {k: v for k,v in enumerate(label_list)}
    label2id = {v: k for k,v in enumerate(label_list)}
num_labels = len(label_list)


print("num_labels=",num_labels)
print("label_list=",label_list)
print("id2label=",id2label)
print("label2id=",label2id)
```

```
num_labels= 7
label_list= ['O', 'B-HEADER', 'I-HEADER', 'B-QUESTION', 'I-QUESTION', 'B-ANSWER', 'I-ANSWER']
id2label= {0: 'O', 1: 'B-HEADER', 2: 'I-HEADER', 3: 'B-QUESTION', 4: 'I-QUESTION', 5: 'B-ANSWER', 6: 'I-ANSWER'}
label2id= {'O': 0, 'B-HEADER': 1, 'I-HEADER': 2, 'B-QUESTION': 3, 'I-QUESTION': 4, 'B-ANSWER': 5, 'I-ANSWER': 6}
```

Next, we'll define a function which we can apply on the entire dataset.

```
def prepare_examples(examples):
  images = examples[image_column_name]
  words = examples[text_column_name]
  boxes = examples[boxes_column_name]
  word_labels = examples[label_column_name]

  encoding = processor(images, words, boxes=boxes, word_labels=word_labels,
                       truncation=True, padding="max_length")

  return encoding


from datasets import Features, Sequence, ClassLabel, Value, Array2D, Array3D

# we need to define custom features for `set_format` (used later on) to work properly
features = Features({
    'pixel_values': Array3D(dtype="float32", shape=(3, 224, 224)),
    'input_ids': Sequence(feature=Value(dtype='int64')),
    'attention_mask': Sequence(Value(dtype='int64')),
    'bbox': Array2D(dtype="int64", shape=(512, 4)),
    'labels': Sequence(feature=Value(dtype='int64')),
})

train_dataset = dataset["train"].map(
    prepare_examples,
    batched=True,
    remove_columns=column_names,
    features=features,
)
eval_dataset = dataset["test"].map(
    prepare_examples,
    batched=True,
    remove_columns=column_names,
    features=features,
)
```

```
Map: 100%                                    149/149 [00:03<00:00, 43.59 examples/s]

Map: 100%                                    50/50 [00:01<00:00, 34.80 examples/s]
```

```
train_dataset
```

```
Dataset({
    features: ['pixel_values', 'input_ids', 'attention_mask', 'bbox', 'labels'],
    num_rows: 149
})
```

```
example = train_dataset[0]
processor.tokenizer.decode(example["input_ids"])
```

```
'<s> R&D : Suggestion: Date: Licensee Yes No 597005708 R&D QUALITY IMPROVEMENT SUGGESTION/ SOLUTION FORM Name / Phone Ex
t. : M. Hamann P. Harper, P. Martinez 9/ 3/ 92 R&D Group: J. S. Wigand Supervisor / Manager Discontinue coal retention a
nalyses on licensee submitted product samples (Note : Coal Retention testing is not performed by most licensees. Other B
&W physical measurements as ends stability and inspection for soft spots in ciparettes are thought to be sufficient meas
ures to assure cigarette physical integrity. The proposed action will increase laboratory productivity . ) Suggested Sol
utions (s) : Delete coal retention from the list of standard analyses performed on licensee submitted product samples. S
pecial requests for coal retention testing could still be submitted on an exception basis. Have you contacted your Manag
er/ Supervisor? Manager Comments: Manager, please contact suggester and forward comments to the Quality Council. qip . w
p</s><pad><pad><pad><pad><pad><pad><pad><…'
```

Next, we set the format to PyTorch.

```
train_dataset.set_format("torch")
```

Let's verify that everything was created properly:

```
import torch

example = train_dataset[0]
for k,v in example.items():
    print(k,v.shape)
```

```
pixel_values torch.Size([3, 224, 224])
input_ids torch.Size([512])
attention_mask torch.Size([512])
bbox torch.Size([512, 4])
labels torch.Size([512])
```

```
eval_dataset
```

```
Dataset({
    features: ['pixel_values', 'input_ids', 'attention_mask', 'bbox', 'labels'],
    num_rows: 50
})
```

```
processor.tokenizer.decode(eval_dataset[0]["input_ids"])
```

```
'<s> TO: DATE: 3 Fax: NOTE: 82092117 614 -466 -5087 Dec 10 '98 17 :46 P. 01 ATT. GEN. ADMIN. OFFICE Attorney General Bet
ty D. Montgomery CONFIDENTIAL FACSIMILE TRANSMISSION COVER SHEET (614) 466- 5087 FAX NO. George Baroody (336) 335- 7392
FAX NUMBER: PHONE NUMBER: (336) 335- 7363 NUMBER OF PAGES INCLUDING COVER SHEET: June Flynn for Eric Brown/ (614) 466- 8
980 SENDER /PHONE NUMBER: SPECIAL INSTRUCTIONS: IF YOU DO NOT RECEIVE ANY OF THE PAGES PROPERLY, PLEASE CONTACT SENDER A
S SOON AS POSSIBLE THIS MESSAGE IS INTENDED ONLY FOR THE USE OF THE INDIVIDUAL OR ENTITY TO WHOM IT IS ADDRESSED AND MAY
CONTAIN INFORMATION THAT IS PRIVILEGED. CONFIDENTIAL, AND EXEMPT FROM DISCLOSURE UNDER APPLICABLE LAW. If the reader of
this message is not the intended recipient of the employee or agent responsible for delivering the message to the intend
ed recipient, you are hereby notified that any dissemination, distribution, copying, or conveying of this communication
in any manner is strictly prohibited. If…'
```

```
for id, label in zip(train_dataset[0]["input_ids"], train_dataset[0]["labels"]):
  print(processor.tokenizer.decode([id]), label.item())
```

```
<pad> -100
<pad> -100
<pad> -100
```

```
        [label_list[l] for (p, l) in zip(prediction, label) if l != -100]
        for prediction, label in zip(predictions, labels)
    ]

    results = metric.compute(predictions=true_predictions, references=true_labels)
    if return_entity_level_metrics:
        # Unpack nested dictionaries
        final_results = {}
        for key, value in results.items():
            if isinstance(value, dict):
                for n, v in value.items():
                    final_results[f"{key}_{n}"] = v
            else:
                final_results[key] = value
        return final_results
    else:
        return {
            "precision": results["overall_precision"],
            "recall": results["overall_recall"],
            "f1": results["overall_f1"],
            "accuracy": results["overall_accuracy"],
        }
```

Define the model

Next we define the model: this is a Transformer encoder with pre-trained weights, and a randomly initialized head on top for token classification.

```
from transformers import LayoutLMv3ForTokenClassification

model = LayoutLMv3ForTokenClassification.from_pretrained("microsoft/layoutlmv3-base",
                                                         id2label=id2label,
                                                         label2id=label2id)
```

```
model.safetensors: 100%                                    501M/501M [00:01<00:00, 264MB/s]
Some weights of LayoutLMv3ForTokenClassification were not initialized from the model checkpoint at microsoft/layoutlmv3-b
You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.
```

Define TrainingArguments + Trainer

Next we define the `TrainingArguments`, which define all hyperparameters related to training. Note that there is a huge amount of parameters to tweak, check the docs for more info.

```
from transformers import TrainingArguments, Trainer

training_args = TrainingArguments(output_dir="test",
                                  max_steps=1000,
                                  per_device_train_batch_size=2,
                                  per_device_eval_batch_size=2,
                                  learning_rate=1e-5,
                                  evaluation_strategy="steps",
                                  eval_steps=100,
                                  load_best_model_at_end=True,
                                  metric_for_best_model="f1")
```

```
/usr/local/lib/python3.10/dist-packages/transformers/training_args.py:1559: FutureWarning: `evaluation_strategy` is depre
  warnings.warn(
```

We can now instantiate a Trainer, with the model and args defined above. We also provide our datasets, as well as a "default data collator" - which will batch the examples using `torch.stack`. We also provide our `compute_metrics` function defined above.

```
from transformers.data.data_collator import default_data_collator

# Initialize our Trainer
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_dataset,
    eval_dataset=eval_dataset,
    tokenizer=processor,
    data_collator=default_data_collator,
    compute_metrics=compute_metrics,
)
```

```
max_steps is given, it will override any value given in num_train_epochs
```

Train the model

Let's train!

```
trainer.train()
```

```
/usr/local/lib/python3.10/dist-packages/transformers/modeling_utils.py:1155: FutureWarning: The `device` argument is depr
  warnings.warn(
```

[1000/1000 07:41, Epoch 13/14]

Step	Training Loss	Validation Loss	Precision	Recall	F1	Accuracy
100	No log	0.645189	0.754203	0.824640	0.787850	0.776061
200	No log	0.527412	0.797521	0.862891	0.828919	0.815524
300	No log	0.506616	0.861553	0.887233	0.874205	0.847260
400	No log	0.534153	0.871435	0.895678	0.883390	0.856294
500	0.570600	0.554360	0.877354	0.902633	0.889814	0.859384
600	0.570600	0.588008	0.869899	0.900149	0.884766	0.859622
700	0.570600	0.641511	0.892910	0.907104	0.899951	0.863069
800	0.570600	0.617226	0.887440	0.912568	0.899829	0.864733
900	0.570600	0.643618	0.890304	0.903130	0.896671	0.862118
1000	0.124300	0.640870	0.889322	0.906110	0.897638	0.861999

```
/usr/local/lib/python3.10/dist-packages/transformers/modeling_utils.py:1155: FutureWarning: The `device` argument is depr
  warnings.warn(
TrainOutput(global_step=1000, training_loss=0.3474191818237305, metrics={'train_runtime': 463.7722,
'train_samples_per_second': 4.312, 'train_steps_per_second': 2.156, 'total_flos': 523783817001984.0, 'train_loss':
0.3474191818237305, 'epoch': 13.333333333333334})
```

Evaluate the model

NOTE: we end up with an F1 score of about 90%. Here's what I got on a typical run:

Step	Training Loss	Val Loss	Precision	Recall	F1	Accuracy
100	No log	0.716025	0.752040	0.824143	0.786442	0.780459
200	No log	0.584986	0.828558	0.876304	0.851762	0.801973
300	No log	0.525926	0.859583	0.900149	0.879398	0.833947
400	No log	0.492821	0.881413	0.904620	0.892866	0.854630

```
500      0.561200    0.528126    0.858382    0.885246    0.871607    0.852490
600      0.561200    0.547107    0.888023    0.906110    0.896976    0.847973
700      0.561200    0.555438    0.887338    0.915549    0.901222    0.859384
800      0.561200    0.582942    0.881471    0.905117    0.893137    0.854749
900      0.561200    0.599762    0.891051    0.910084    0.900467    0.852015
1000    0.133400    0.608207    0.887222    0.910581    0.898750    0.847855
```

However, this score cannot be directly compared to LayoutLM and LayoutLMv2, as LayoutLMv3 employs so-called **segment position embeddings** (inspired by StructuralLM). This means that several tokens that belong to the same "segment" (let's say, an address) get the same bounding box coordinates, and in return the same 2D position embeddings.

This is also mentioned in the paper:

> Note that LayoutLMv3 and StructuralLM use segment-level layout positions, while the other works use word-level layout positions. The use of segment-level positions may benefit the semantic entity labeling task on FUNSD [25], so the two types of work are not directly comparable.

```
trainer.evaluate()
```

```
***** Running Evaluation *****
  Num examples = 50
  Batch size = 2
/usr/local/lib/python3.7/dist-packages/transformers/modeling_utils.py:813: FutureWarning: The `device` argument is deprec
  "The `device` argument is deprecated and will be removed in v5 of Transformers.", FutureWarning
[25/25 00:03]
{'epoch': 13.33,
 'eval_accuracy': 0.8478545108760251,
 'eval_f1': 0.8987496935523414,
 'eval_loss': 0.6082074642181396,
 'eval_precision': 0.8872216844143272,
 'eval_recall': 0.910581222056632,
 'eval_runtime': 3.4127,
 'eval_samples_per_second': 14.651,
 'eval_steps_per_second': 7.326}
```

Inference

You can load the model for inference as follows:

```
from transformers import AutoModelForTokenClassification

model = AutoModelForTokenClassification.from_pretrained("/content/test/checkpoint-1000")
```

```
  "bos_token_id": 0,
  "classifier_dropout": null,
  "coordinate_size": 128,
  "eos_token_id": 2,
  "has_relative_attention_bias": true,
  "has_spatial_attention_bias": true,
  "hidden_act": "gelu",
  "hidden_dropout_prob": 0.1,
  "hidden_size": 768,
  "id2label": {
    "0": "O",
    "1": "B-HEADER",
    "2": "I-HEADER",
```

```
    "intermediate_size": 3072,
    "label2id": {
      "B-ANSWER": 5,
      "B-HEADER": 1,
      "B-QUESTION": 3,
      "I-ANSWER": 6,
      "I-HEADER": 2,
      "I-QUESTION": 4,
      "O": 0
    },
    "layer_norm_eps": 1e-05,
    "max_2d_position_embeddings": 1024,
    "max_position_embeddings": 514,
    "max_rel_2d_pos": 256,
    "max_rel_pos": 128,
    "model_type": "layoutlmv3",
    "num_attention_heads": 12,
    "num_channels": 3,
    "num_hidden_layers": 12,
    "pad_token_id": 1,
    "patch_size": 16,
    "rel_2d_pos_bins": 64,
    "rel_pos_bins": 32,
    "second_input_size": 112,
    "shape_size": 128,
    "text_embed": true,
    "torch_dtype": "float32",
    "transformers_version": "4.21.0.dev0",
    "type_vocab_size": 1,
    "visual_embed": true,
    "vocab_size": 50265
  }

loading weights file /content/test/checkpoint-1000/pytorch_model.bin
All model checkpoint weights were used when initializing LayoutLMv3ForTokenClassification.

All the weights of LayoutLMv3ForTokenClassification were initialized from the model checkpoint at /content/test/checkpo
```

Let's take an example of the training dataset to show inference.

```
example = dataset["test"][0]
print(example.keys())
```

```
dict_keys(['id', 'tokens', 'bboxes', 'ner_tags', 'image'])
```

We first prepare it for the model using the processor.

```
image = example["image"]
words = example["tokens"]
boxes = example["bboxes"]
word_labels = example["ner_tags"]

encoding = processor(image, words, boxes=boxes, word_labels=word_labels, return_tensors="pt")
for k,v in encoding.items():
  print(k,v.shape)
```

```
input_ids torch.Size([1, 360])
attention_mask torch.Size([1, 360])
bbox torch.Size([1, 360, 4])
labels torch.Size([1, 360])
pixel_values torch.Size([1, 3, 224, 224])
```

Next, we do a forward pass. We use torch.no_grad() as we don't require gradient computation.

```
with torch.no_grad():
  outputs = model(**encoding)
```

```
/usr/local/lib/python3.7/dist-packages/transformers/modeling_utils.py:813: FutureWarning: The `device` argument is deprec
  "The `device` argument is deprecated and will be removed in v5 of Transformers.", FutureWarning
```

The model outputs logits of shape (batch_size, seq_len, num_labels).

```
logits = outputs.logits
logits.shape
```

```
torch.Size([1, 360, 7])
```

We take the highest score for each token, using argmax. This serves as the predicted label for each token.

```
predictions = logits.argmax(-1).squeeze().tolist()
print(predictions)
```

```
[0, 3, 4, 3, 3, 4, 5, 3, 3, 4, 3, 4, 0, 0, 0, 5, 6, 6, 6, 6, 6, 6, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
```

Let's compare this to the ground truth: note that many labels are -100, as we're only labeling the first subword token of each word.

NOTE: at "true inference" time, you don't have access to labels, see the latest section of this notebook how you can use `offset_mapping` in that case.

```
labels = encoding.labels.squeeze().tolist()
print(labels)
```

```
[-100, 3, -100, 3, -100, -100, 5, 3, -100, -100, 3, -100, 0, -100, -100, 5, -100, 6, -100, 6, -100, -100, 0, 0, 0, -100,
```

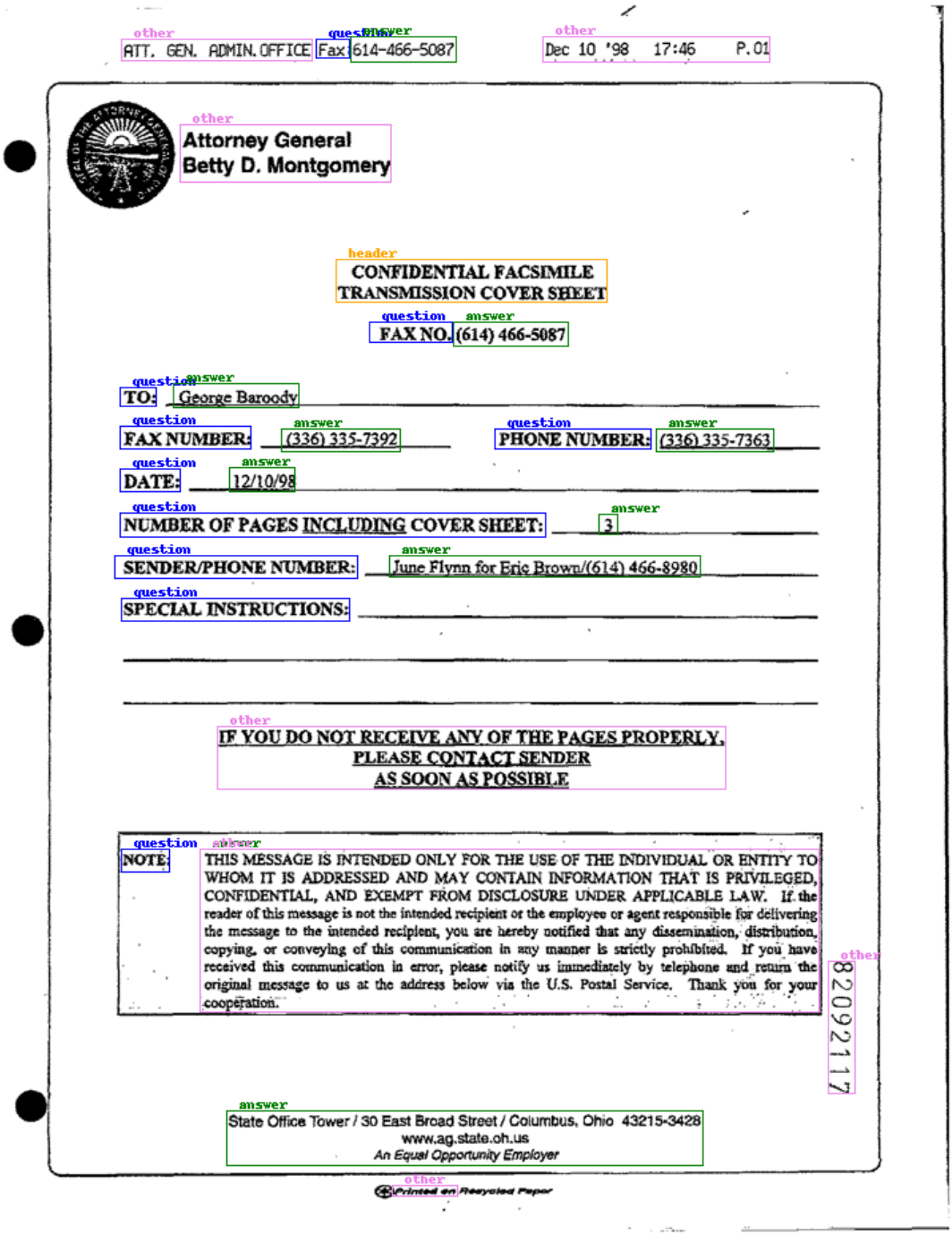
So let's only compare predictions and labels at positions where the label isn't -100. We also want to have the bounding boxes of these (unnormalized):

```
def unnormalize_box(bbox, width, height):
     return [
         width * (bbox[0] / 1000),
         height * (bbox[1] / 1000),
         width * (bbox[2] / 1000),
         height * (bbox[3] / 1000),
     ]

token_boxes = encoding.bbox.squeeze().tolist()
width, height = image.size

true_predictions = [model.config.id2label[pred] for pred, label in zip(predictions, labels) if label != - 100]
true_labels = [model.config.id2label[label] for prediction, label in zip(predictions, labels) if label != -100]
true_boxes = [unnormalize_box(box, width, height) for box, label in zip(token_boxes, labels) if label != -100]
```

```
from PIL import ImageDraw, ImageFont

draw = ImageDraw.Draw(image)

font = ImageFont.load_default()

def iob_to_label(label):
    label = label[2:]
    if not label:
      return 'other'
    return label

label2color = {'question':'blue', 'answer':'green', 'header':'orange', 'other':'violet'}
```

```
for prediction, box in zip(true_predictions, true_boxes):
    predicted_label = iob_to_label(prediction).lower()
    draw.rectangle(box, outline=label2color[predicted_label])
    draw.text((box[0] + 10, box[1] - 10), text=predicted_label, fill=label2color[predicted_label], font=font)
```

```
image
```

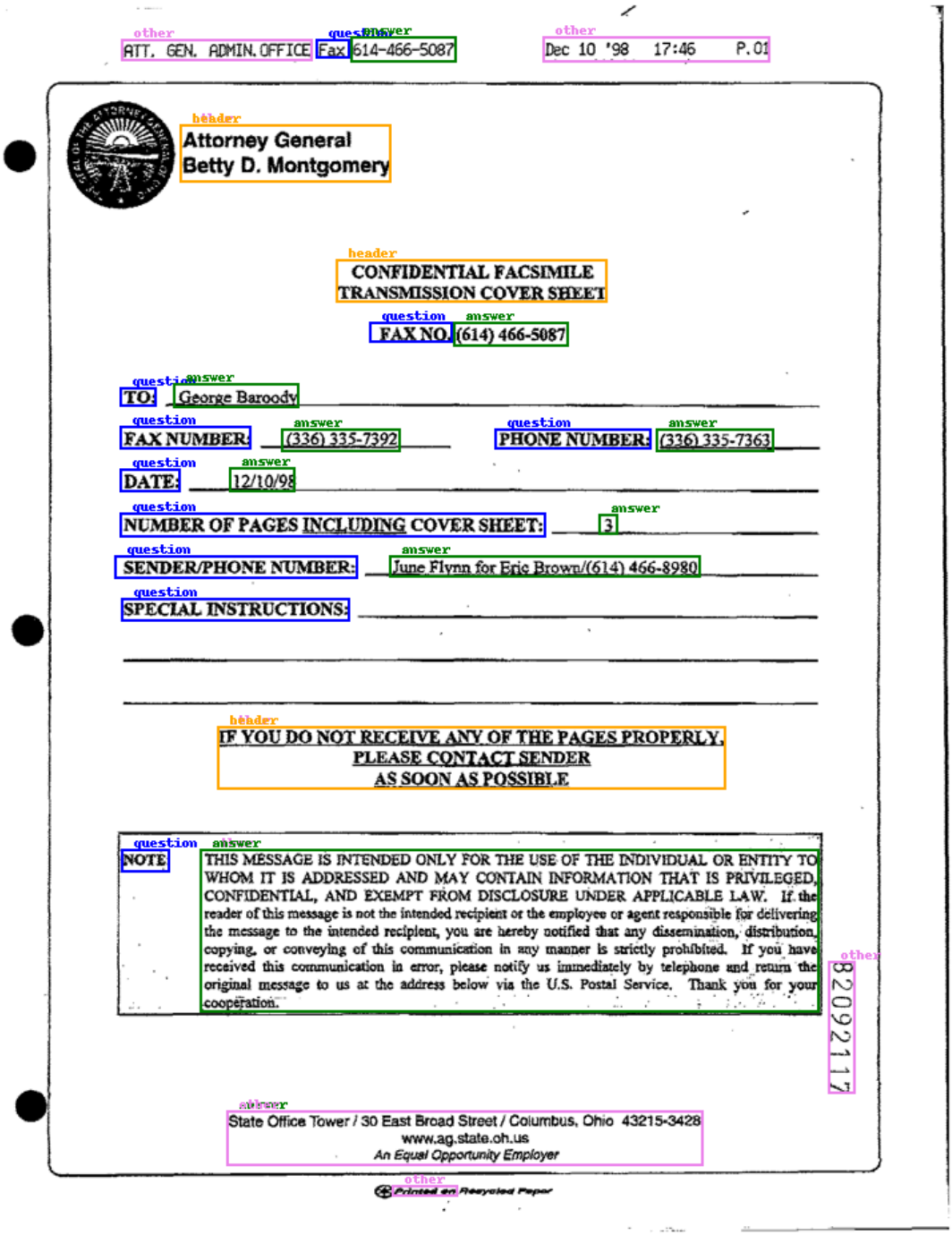


Compare this to the ground truth:

```
image = example["image"]
image = image.convert("RGB")

draw = ImageDraw.Draw(image)
```

```
for word, box, label in zip(example['tokens'], example['bboxes'], example['ner_tags']):
  actual_label = iob_to_label(id2label[label]).lower()
  box = unnormalize_box(box, width, height)
  draw.rectangle(box, outline=label2color[actual_label], width=2)
  draw.text((box[0] + 10, box[1] - 10), actual_label, fill=label2color[actual_label], font=font)
```

```
image
```



Note: inference when you don't have labels

The code above used the `labels` to determine which tokens were at the start of a particular word or not. Of course, at inference time, you don't have access to any labels. In that case, you can leverage the `offset_mapping` returned by the tokenizer. I do have a notebook

for that (for LayoutLMv2, but it's equivalent for LayoutLMv3) here.

Start coding or generate with AI.